Universidad Politécnica Metropolitana del Estado de Hidalgo (UPMH

Maestría en Inteligencia Artificial

Algoritmos Genéticos

Representación de individuos, operadores genéticos, implementación en Python y ejercicios prácticos

Materia: Algoritmos Evolutivos
Alumna: Jessica Melani Romero Lora
Profesor: Dr. Jaime Aguilar Ortiz

Octubre 2026

1. Definiciones prácticas

- **Algoritmo Genético (AG):** Método de optimización metaheurístico que simula la evolución natural. Una población de soluciones candidatas compite, se cruza y muta hasta converger hacia el óptimo.
- **Cromosoma (individuo):** Estructura de datos que codifica una solución al problema. Es la unidad sobre la que actúan los operadores genéticos.
- **Gen:** Cada posición del cromosoma. Almacena un valor (bit, número real o índice) que representa una variable de decisión.
- **Fitness (aptitud):** Valor numérico que mide la calidad de un individuo. El AG lo usa para decidir quién se reproduce y quién no.
- **Generación:** Una iteración completa del ciclo: evaluación → selección → cruzamiento → mutación → nueva población.

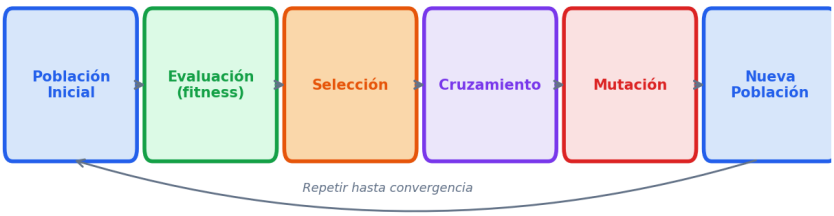


Figura 1. Ciclo iterativo de un algoritmo genético.

2. Representación y codificación

Elegir cómo codificar las soluciones es el primer paso y condiciona el rendimiento del AG. La tabla muestra los esquemas más usados:

Tipo	Representación	Uso típico	Ejemplo
Binaria	Cadena de 0 y 1	Optimización discreta	[1,0,1,1,0]
Real	Números decimales	Variables continuas	[3.14, 0.72, 5.91]
Permutación	Secuencia sin repetir	Rutas (TSP)	[3,1,5,2,7,4,6]
Entera	Números enteros	Selección de categorías	[2, 5, 1, 3]

Tabla 1. Esquemas de codificación y sus aplicaciones.

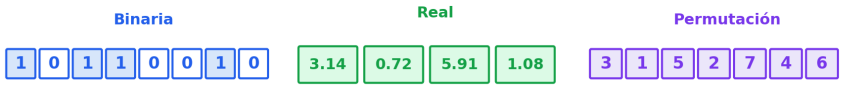


Figura 2. Representación gráfica de cromosomas binario, real y de permutación.

Ejemplo práctico: para maximizar $f(x) = x^2$ en [0, 31], codificamos x con 5 bits ($2^5 = 32$ valores). El cromosoma [1,0,1,1,0] equivale a 22_{10}, por lo que fitness = f(22) = 484.

3. Operadores genéticos

3.1 Selección

Elige a los individuos que se reproducirán. En la selección por ruleta, la probabilidad de cada individuo es proporcional a su fitness. En la selección por torneo, se eligen k candidatos al azar y gana el de mayor aptitud.

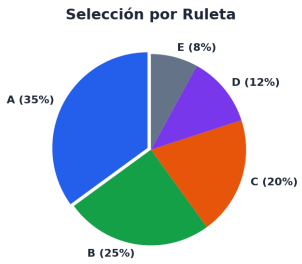


Figura 3. Ruleta proporcional al fitness.

3.2 Cruzamiento (crossover)

Combina segmentos de dos padres para generar hijos. El cruzamiento de un punto elige una posición aleatoria e intercambia las colas de ambos cromosomas. También existen los de dos puntos y el uniforme.

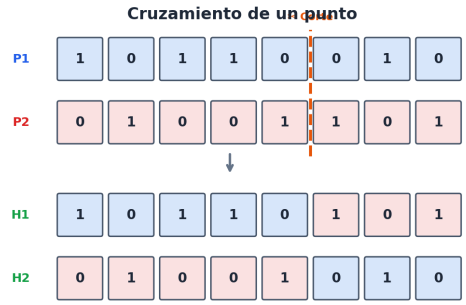


Figura 4. Cruzamiento de un punto con corte en la posición 5.

3.3 Mutación

Altera aleatoriamente uno o más genes de un individuo. Mantiene la diversidad genética y permite escapar de óptimos locales. La tasa habitual va de 0.001 a 0.05 por gen.

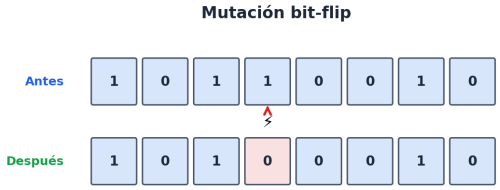


Figura 5. Mutación bit-flip: el gen en la posición 4 cambia de 1 a 0.

4. Implementación en Python

A continuación se muestra un AG mínimo para maximizar f(x) = x² con codificación binaria de 5 bits:

```
import random

POP, BITS, GEN, PC, PM = 20, 5, 50, 0.8, 0.05

def fitness(ind):                          # Decodifica y evalúa
    x = int(''.join(map(str,ind)), 2)
    return x ** 2

def seleccion(pob, fits):                  # Ruleta
    total = sum(fits)
    r = random.uniform(0, total)
    acc = 0
    for ind, f in zip(pob, fits):
        acc += f
        if acc >= r: return ind[:]

def cruzamiento(p1, p2):                   # Un punto
    if random.random() < PC:
        pt = random.randint(1, BITS - 1)
        return p1[:pt]+p2[pt:], p2[:pt]+p1[pt:]
    return p1[:], p2[:]

def mutacion(ind):                         # Bit-flip
    return [b if random.random()>PM else 1-b for b in ind]

# --- Bucle principal ---
pob = [[random.randint(0,1) for _ in range(BITS)] for _ in range(POP)]
for g in range(GEN):
    fits = [fitness(i) for i in pob]
    nueva = []
    while len(nueva) < POP:
        p1 = seleccion(pob, fits)
        p2 = seleccion(pob, fits)
        h1, h2 = cruzamiento(p1, p2)
        nueva += [mutacion(h1), mutacion(h2)]
    pob = nueva[:POP]
    print(f'Gen {g}: mejor = {max(fits)}')
```

Código 1. Algoritmo genético completo en Python (~30 líneas).

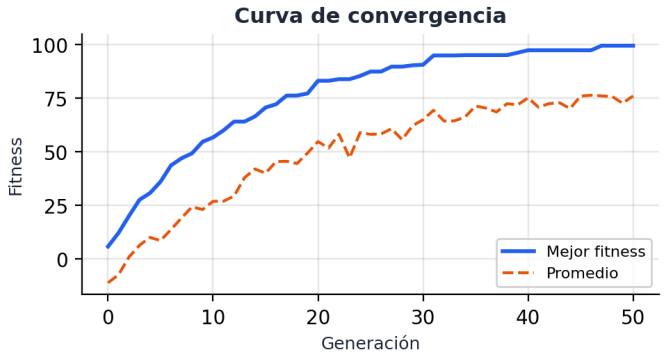


Figura 6. Curva de convergencia típica: el fitness mejora rápidamente y se estabiliza.

5. Ejercicios propuestos

Ejercicio 1 — Decodificación binaria. Dado el cromosoma [1,1,0,1,0], calcule el valor decimal y evalúe f(x) = x². Respuesta: 1·16 + 1·8 + 0·4 + 1·2 + 0·1 = 26, f(26) = 676.

Ejercicio 2 — Cruzamiento manual. Sean P1 = [1,1,1,0,0] y P2 = [0,0,0,1,1] con punto de corte en la posición 3. Determine H1 y H2. Respuesta: H1 = [1,1,1,1,1] (x = 31, f = 961), H2 = [0,0,0,0,0] (x = 0, f = 0).

Ejercicio 3 — Probabilidad de selección. Una población tiene individuos con fitness 100, 200, 300 y 400. Calcule la probabilidad de cada uno en la selección por ruleta. Respuesta: 10%, 20%, 30% y 40% respectivamente (total = 1000).

Ejercicio 4 — Modifique el código. Cambie la función de fitness a f(x) = x·sin(x) y aumente la codificación a 8 bits ([0, 255]). Observe cómo cambia la convergencia y compare con el caso original.